

作业感受（第三次挑战）

学号 0102603133

（1）简述下你是如何完成第三次挑战的（如何使用 AI、如何工作迭代等）

第三次挑战我大致是按「先搭环境、再跑通默认配置、然后逐组对照、最后改进」的顺序做的。环境上预约了同一台机器的四张 4090D，把 SGLang 和 Ray 装成两个独立的 Python 环境、只通过 HTTP 通信，模型用 Qwen3-0.6B，四个后端分别绑一张卡。装依赖时踩了不少坑：默认 pip 源只有零点几 MB/s，实测换了清华源之后快了二十多倍；Ray 2.56 对 protobuf 的版本也有硬性要求，来回试了几次才稳定下来。

跑组 A 的时候发现吞吐只有 42 req/s、TTFT p95 将近 20 秒，看数据才发现是默认 max_ongoing_requests=5 太小，请求全堵在 Serve 侧排队。B 组扫了 16 和 32 两个值：16 吞吐最高且延迟最低，32 反而因为 decode 批变大出现队头阻塞，于是选定 16。C 组用一致性哈希做严格前缀亲和，缓存命中率确实涨到 95.78%、prefill token 降到三分之一，但负载里那个超热前缀族把 62.5% 的请求都砸到一个副本上，吞吐反而腰斩——这一组让我直观看到「缓存收益是每副本局部的，吞吐由最慢副本决定」。D 组我自己写了前缀亲和优先、负载高时溢出的路由器，结果首跑只有 5 req/s、四成请求失败，对照 Ray 内置路由器的源码逐层排查，最后发现是自定义路由器少了 FIFOMixin、丢了 FIFO 排队语义，补上之后立刻恢复到 63 req/s。整个过程迭代了五轮，每轮都先重部署、清空四个后端缓存再回放，保证组间可比。

（2）你觉得哪部分工作最困难，你是如何克服这个困难的

最卡住的地方是组 D 那次故障。路由逻辑本身并不复杂，写完一跑，405 条请求失败、吞吐只有 5 req/s，TTFT p95 一百多秒，比什么都不做还差。我一开始以为是并发上限给小了，改完再跑还是老样子；又去翻 proxy 的日志，报的是读队列长度超时，于是把副本里统计在飞数的同步方法改成异步，快了一点，但仍然有四成请求失败。真正找到原因是很后面的事了：我把 Ray 自带的 p2c 路由器源码翻出来，看到它的类声明是继承 FIFOMixin、LocalityMixin、MultiplexMixin，才反应过来我自己那个只继承了 RequestRouter——Ray 内置的 FIFO 排队语义是靠 FIFOMixin 带进来的，缺了它，2048 条高并发下路由任务就退化。补上之后同一份代码立刻从 5 req/s 回到 63 req/s，2048 条全成功。这件事教会我的排查方法是：框架里出现讲不通的行为时，别继续猜参数，把官方自带的实现当参照物，逐个比对继承链和返回值语义，差别往往就在一两行里。

第二个麻烦是组 C 的结果让我怀疑自己是不是哪里配错了：缓存命中率 95.78%，是五组里最高的，吞吐却只有 30.98 req/s，排最后。后来把后端请求分布打出来才看明白，SGLang-0 扛了 1280/2048，也就是 62.5%，因为负载里那个超热前缀族连同哈希到同一副本的其它族全砸过去了。我一开始觉得是实验做坏了，其实数据是对的，是我的直觉错了——命中率高和系统快根本不是一回事。

（3）类似的从 0 到 1 的科研工作需要包含哪些方面的研究，对你今后的科研工作有什么启发

要说从 0 到 1

做这类工作要准备什么，我的体会是先得把测量本身做扎实：固定负载、每轮开始前清空四个后端缓存、软件版本记清楚，不然两轮数据根本没法比。然后需要一个基线和单变量扫描，这次就是先测默认配置，再只动 max_ongoing_requests 一个参数扫 16 和 32，才发现 32 反而更差（decode 批变大导致队头阻塞）；要是能一次改好几个参数，就看不出到底哪一项在起作用。还需要一个能推翻自己直觉的对照组，C 组就扮演了这个角色，它的缓存命中率最高 却吞吐最低。

对我后面做科研的启发主要是两条。一是评价一个策略要看它在最慢那个副本上表现如何，而不是看平均值——这次所有

"系统级"的结论最后都归结到"热点副本成了瓶颈"这一件事上。二是别太信任某一个指标：命中率、吞吐、TTFT p95 经常给出互相矛盾的排序，把它们放在一起看才有意义。另外我也打算以后把失败记录一并写进 README，这次 FIFOMixin 的坑如果不记下来，下次大概率还会再踩一遍。